# Practical – 6

## Write a program to implement the Naïve Bayesian classifier for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets.

### 1. What is Naïve Bayes?

Naïve Bayes is a supervised machine learning algorithm used for classification problems.
It is based on Bayes Theorem, which calculates the probability of a class based on prior knowledge of conditions related to that class.

The algorithm assumes that the features are independent of each other, which is why it is called "Naïve".

Naïve Bayes is widely used for text classification, spam filtering, and medical diagnosis.

### 2. What is Bayes Theorem?

Bayes Theorem describes the probability of an event based on prior knowledge.

Formula

P(A|B)=P(B|A)×P(A)P(B)P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}P(A|B)=P(B)P(B|A)×P(A) Where:

- P(A|B) = Posterior Probability
- P(B|A) = Likelihood
- P(A) = Prior Probability
- P(B) = Evidence

In machine learning, this formula helps to predict the class label of new data.

### 3. Core Concept of Naïve Bayes

Naïve Bayes calculates the probability of each class and chooses the class with the highest probability.

Posterior Probability

P(Class|Features)=P(Features|Class)×P(Class)P(Features)P(Class|Features) = \frac{P(Features|Class) \times P(Class)}{P(Features)}P(Class|Features)=P(Features)P(Features|Class)×P(Class)

Where:

- P(Class) = Prior probability of the class
- P(Features|Class) = Probability of features given the class
- P(Class|Features) = Probability of class given features

The classifier predicts the class that has the maximum posterior probability.

**4. Types of Naïve Bayes**

There are mainly three types of Naïve Bayes classifiers:

1. Gaussian Naïve Bayes

Used when the features follow a normal distribution.

2. Multinomial Naïve Bayes

Used mainly for text classification problems such as spam detection.

3. Bernoulli Naïve Bayes

Used when features are binary (0 or 1).

**5. Steps for Naïve Bayes Implementation**

I.: Import Libraries

```
import kagglehub
path = kagglehub.dataset_download("amineipad/titanic-dataset")
print("Path to dataset files:", path)
```

II. : Load Dataset from CSV File

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB from
sklearn.metrics import accuracy_score from
sklearn.preprocessing import LabelEncoder
data = pd.read_csv(path + "/train.csv")
print("First 5 rows:") print(data.head())
```

OUTPUT:

```
••• First 5 rows:
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S
```

III.: Separate Features and Target data

```
data[['Survived','Pclass','Sex','Age','SibSp','Parch','Fare','Embarked']]
```

IV. Handle Missing Values

```
data['Age'].fillna(data['Age'].mean(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0],
inplace=True)
```

V. Convert Categorical Data to Numeric

```
le = LabelEncoder()
data['Sex'] = le.fit_transform(data['Sex'])
data['Embarked'] = le.fit_transform(data['Embarked'])
```

VI.: Split Training and Testing Data

```
X = data.drop('Survived', axis=1)
y = data['Survived']
```

VII.: Train Naïve Bayes Model

```
model = GaussianNB()
model.fit(X_train, y_train)
```

VIII.: Make Predictions

```
y_pred=model.predict(X_test)
print("Predicted values:")
print(y_pred)
```

OUTPUT:

```
Predicted values:
[0 0 0 1 1 1 1 0 1 1 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 1 0 0 1 0 1 1 1 0 0 0
 1 1 0 0 0 0 0 1 0 0 0 0 1 1 1 0 1 0 1 0 1 1 1 0 1 1 0 0 1 0 0 0 1 1 1 1 1
 0 0 1 1 1 1 0 1 1 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 1 0 0 1 1
 0 1 0 1 0 1 0 1 1 0 1 1 0 0 1 1 1 1 0 1 0 0 1 1 1 1 0 0 1 0 1 0 0 0 1 0 0
 1 0 0 0 0 1 0 0 0 1 1 1 0 0 0 1 0 1 0 1 0 0 1 1 0 1 0 0 0 1 1]
```

IX.: Calculate Accuracy accuracy = accuracy_score(y_test, y_pred)

print("Accuracy of Naive Bayes Classifier:", accuracy * 100)

OUTPUT:

```
Accuracy of Naive Bayes Classifier: 77.6536312849162
```

X.Test with new sample sample =

[[3,1,25,0,0,7.25,2]] prediction =

model.predict(sample)

print("Prediction:", prediction)

OUTPUT:

```
Prediction: [0]
```

**6. Advantages of Naïve Bayes**

- Simple and easy to implement
- Works well with large datasets
- Requires less training data
- Fast prediction speed
- Performs well in text classification problems

**7. Limitations of Naïve Bayes**

- Assumes features are independent
- Not suitable when features are highly correlated
- Zero probability problem may occur if a category is missing in training data

**8. Applications of Naïve Bayes**

- Spam email detection

- Sentiment analysis
- Document classification
- Medical diagnosis
- Recommendation systems

**Conclusion**

Naïve Bayes is a simple yet powerful classification algorithm based on Bayes theorem. It assumes independence between features and calculates probabilities to classify new data. Due to its simplicity and efficiency, it is widely used in machine learning applications such as spam filtering, text classification, and medical prediction.